



Projeto 2, Programação II

```
-----  
                                MENU  
-----  
1 - Planear viagem  
2 - Operações com rotas  
3 - Operações com portos  
4 - Sair  
-----  
Resposta: 
```

Trabalho realizado por:

- . Gonçalo Miguel Rainho Taborda, nº14065, LEIC
- . Henrique Lima Domingos, nº14064, LEIC
- . Manuel Correia da Silva Roque, nº14061, LEIC

Professora:

- . Valéria Magalhães Pequeno

Datas:

- . Relatório-> 24/05/2024
- . Código-> 22/05/2024



2- Introdução:

- ➔ No presente relatório, iremos discutir o desenvolvimento do nosso projeto e os aspetos abordados durante o decorrer do mesmo.
- ➔ Inicialmente, adotamos uma abordagem metodológica Top-Down para a elaboração do código. Essa metodologia envolveu a identificação e organização dos requisitos do projeto em tópicos antes de iniciar a implementação do código propriamente dito. Esse procedimento visou garantir que a lógica do código fosse concebida de forma precisa e eficiente. O código foi desenvolvido utilizando a aplicação Visual Studio Code (VSC) e foi implementado na linguagem de programação Python. A escrita do código foi guiada pelas diretrizes estabelecidas no enunciado fornecido pela professora.
- ➔ O desenvolvimento do código foi realizado de acordo com as melhores práticas de programação, buscando sempre a correção e melhoria da lógica implementada.

3- Estrutura de dados:

- ➔ Inicialmente criamos funções, compostas por grafos para definir as rotas estabelecidas na imagem que a professora disponibilizou.
- ➔ Já no “corpo principal” da função, começamos a dinamizar a estrutura do nosso código. Começamos por perguntar ao utilizador o porto de origem e o porto de chegada, depois disso, perguntamos se ele na sua viagem deseja fazer quaisquer paragens obrigatórias. Se a resposta do mesmo fosse que sim, teria de escolher o número dos portos em que iria parar e o respetivo nome.
Por fim, o utilizador teria de indicar quantos dias iria demorar o seu percurso e através da sua resposta, calcular-se-ia o gasto total. Caso tivéssemos a falar de navios de carga, o seu gasto final, apontaria para um possível caminho mais curto.
- ➔ Atendendo a isto, precisámos de implementar uma função que nos retirasse do ficheiro Excel, os navios de carga e de passageiros, visto que no Excel existia mais do que dois tipos de navios, tivemos de utilizar bibliotecas do Python, estas indicadas pela professora, para limparmos esse “lixo” e apenas aloarmos na memória o que precisávamos.

4 – Algoritmos:

Pseudocódigo da “função criar_grafo_carga”

Função criar_grafo_carga()

 Iniciar grafo direcionado chamado carga

 Adicionar todas as cidades ao grafo carga

 Definir atributos dos nós do grafo com base nas cidades

 Para cada (origem, destino, dias) em Rotas_carga

 Adicionar aresta de origem para destino com peso igual a dias

 Adicionar aresta de destino para origem com peso igual a dias

 Retornar grafo carga

Fim da função

Pseudocódigo da função “criar_grafo_passageiros”

Função criar_grafo_passageiros()

 Iniciar grafo direcionado chamado passageiros

 Adicionar todas as cidades ao grafo passageiros

 Definir atributos dos nós do grafo com base nas cidades

 Para cada (origem, destino, dias) em Rotas_passageiro

 Adicionar aresta de origem para destino com peso igual a dias

 Adicionar aresta de destino para origem com peso igual a dias

 Retornar grafo passageiros

Fim da função

Pseudocódigo da Função “caminho_mais_curto”

Função caminho_mais_curto(grafo, origem, destino, dias_max)

Tente

 Encontrar caminho mais curto no grafo de origem para destino considerando o peso

 Calcular total de dias do caminho

 Calcular custo total da viagem

Se dias for maior que dias_max

 Exibir mensagem de que o tempo é insuficiente

Retornar [caminho, dias, custo]

Se não houver caminho

 Exibir mensagem de que não existe caminho

Retornar lista vazia

Fim da função

Pseudocódigo da Função “caminhos_todos”

Função caminhos_todos(grafo, origem, destino, dias_max)

Tente

Encontrar todos os caminhos simples de origem para destino dentro de dias_max

Iniciar lista vazia chamada caminhos_completos

Para cada caminho em caminhos

Calcular total de dias do caminho

Calcular custo total da viagem

Adicionar (caminho, dias, custo) a caminhos_completos

Retornar caminhos_completos

Se não houver caminho

Retornar lista vazia

Fim da função

Pseudocódigo da função “procurar_navio”

Função procurar_navio(nome_navio)

Definir tipo como None

Definir encontrado como False

Enquanto não encontrado

Tente

Ler dados do Excel para navios de passageiros

Filtrar dados pelo nome do navio

Se houver resultado

Exibir mensagem de navio de passageiros encontrado

Retornar 1

Ler dados do Excel para navios de carga

Filtrar dados pelo nome do navio

Se houver resultado

Exibir mensagem de navio de carga encontrado

Retornar 2

Exibir mensagem de navio não encontrado

Quebrar o loop

Se arquivo não encontrado

Exibir mensagem de arquivo não encontrado

Se ocorrer outro erro

Exibir mensagem de erro

Retornar 0

Retornar tipo

Fim da função

Pseudocódigo da função “porto_origem”

Função porto_origem()

 Enquanto True

 Solicitar ao usuário o porto de origem

 Se porto_origem estiver em Cidades

 Retornar porto_origem

 Caso contrário

 Exibir mensagem de cidade indisponível

Fim da função

Pseudocódigo da função “porto_chegada”

Função porto_chegada()

Enquanto True

 Solicitar ao usuário o porto de chegada

 Se porto_chegada estiver em Cidades

 Retornar porto_chegada

 Caso contrário

 Exibir mensagem de cidade indisponível

Fim da função

Pseudocódigo da função “obter_escala”

Função obter_escala(origem)

Enquanto True

 Perguntar ao usuário se deseja fazer escala

 Se resposta for 's'

 Iniciar lista escala vazia

 Solicitar ao usuário o número de escalas

 Enquanto i não for igual ao número de escalas

 Solicitar ao usuário o nome da escala i

 Se escala estiver disponível e não for igual à origem

 Adicionar escala à lista

 Incrementar i

 Caso contrário

 Exibir mensagem de escala indisponível ou igual à origem

 Retornar escala

 Se resposta for 'n'

 Exibir mensagem de viagem sem escala

 Retornar None

 Caso contrário

 Exibir mensagem de resposta inválida

Fim da função

Pseudocódigo da função “dias”

Função dias()

Enquanto True

 Solicitar ao usuário o número de dias disponíveis

 Se dias for um número positivo

 Retornar dias

 Caso contrário

 Exibir mensagem de número inválido

Fim da função

Pseudocódigo da função “tipo_navio”

Função tipo_navio(nome_navio)

 Enquanto True

 Chamar função procurar_navio(nome_navio)

 Se tipo_navio encontrado for 1

 Exibir mensagem de navio de passageiros encontrado

 Retornar 1

 Se tipo_navio encontrado for 2

 Exibir mensagem de navio de carga encontrado

 Retornar 2

 Caso contrário

 Exibir mensagem de navio não encontrado

 Retornar 0

Fim da função

Pseudocódigo da função "gerar_pdf"

Função gerar_pdf(caminhos)

Cria um objeto Canvas para o PDF

Definir ficheiro como um novo Canvas chamado "orcamento_viagem.pdf" com tamanho de página 'letter'

Define a largura e altura da página

Definir largura e altura com os valores da constante 'letter'

Escreve o título no PDF

Desenhar string "Orçamento de Viagem" na posição (100, altura- 40)

Definir y_position como altura- 80

Itera sobre os caminhos fornecidos

Para cada caminho com índice (posição, caminho) em enumerate(caminhos, começando de 1)

Desenha o título do caminho

Desenhar string "Caminho {posição}:" na posição (100, y_position)

y_position -= 20

Desenha os detalhes do caminho

Para cada par (chave, valor) no dicionário caminho

Desenhar string "{chave.capitalize()}: {valor}" na posição (120, y_position)

y_position -= 20

y_position -= 10

Verifica se a posição Y é menor que 40 para criar uma nova página

Se y_position < 40

Chamar método showPage() do ficheiro

Redefinir y_position como altura- 40

Salva o PDF

Chamar método save() do ficheiro

Fim da função

5- Conclusão:

Estamos confiantes de que alcançamos os objetivos estabelecidos, conforme as diretrizes definidas. Durante o desenvolvimento do projeto, confrontamo-nos com uma série de desafios distintos, tais como:

⇧ Realizar funções utilizando grafos.

⇧ Utilizarmos as bibliotecas do Python corretamente e sabermos exatamente o que poderíamos beneficiar com elas no desenvolvimento do código.

⇧ Modularização da forma mais rigorosa e completa do nosso código, pois só assim é que conseguimos organizar os nossos ideais e tornar tudo mais simples, numa perspectiva de utilização do mesmo.

Contudo, o nosso balanço relativamente a este projeto é positivo pois conseguimos gerir bem o tempo que tínhamos para realizar as tarefas que nos foram propostas, com o máximo de eficácia e de concentração possível.

Em resumo, este relatório proporcionou uma visão detalhada do processo de desenvolvimento do projeto, desde a análise inicial dos requisitos até a implementação das funcionalidades específicas de cada ponto deste trabalho. Esperemos que este documento não apenas forneça uma compreensão clara do trabalho realizado, mas também destaque o papel crucial da metodologia adotada, das escolhas feitas durante a realização do código e as principais funcionalidades implementadas no mesmo.

6- Webgrafia:

➔ https://enautica-my.sharepoint.com/personal/valeriapequeno_enautica_pt/_layouts/15/onedrive.aspx?id=%2Fpersonal%2Fvaleriapequeno%5Fenautica%5Fpt%2FDocuments%2Fdisciplinas%2Fprogramacao2%5FLEIC%2FmaterialTeorico%2Fmodulo07%5Fgrafos%5Fvmp%2Epdf&parent=%2Fpersonal%2Fvaleriapequeno%5Fenautica%5Fpt%2FDocuments%2Fdisciplinas%2Fprogramacao2%5FLEIC%2FmaterialTeorico&ga=1

➔ <https://www.youtube.com/watch?v=XKHEtdqhLK8&t=10133s>

➔ <https://www.w3schools.com/python/default.asp>

➔ Livros: Programação em Python, Ernesto Costa

⇧ https://www.fnac.pt/Programacao-em-Python-Ernesto-Costa/a935250?origin=google_pla_livro&gad_source=1&gclid=Cj0KCQjw3ZayBhDRARIsAPWzx8r6tzaN1xQoYnw2GSRNqbFsKh8izns_CzMTHP4SpCtX87D1X3PrHewaAm52EALw_wcB